

FINAL REPORT

ARTIFICIAL INTELLIGENCE

Hammad Amer
Shayaan Khalid

22i-0877
22i-0863

The goal of this project was to develop a controller for a simulated race car in the TORCS environment, capable of autonomous operation while allowing manual intervention. Implemented in Python, the controller interfaces with the TORCS server using UDP communication through the snakeoil.py library. The design combines a deep learning model for AI-driven control, rule-based logic for gear shifting, and keyboard inputs for manual control, providing flexibility and reliability during races.

The controller is implemented in the Driver class, which manages the car's state and control actions. The CarState class stores sensor data, including speed, RPM, track position, and opponent distances, while the CarControl class handles actuation parameters such as acceleration, braking, steering, and gear.

The system supports two operational modes:

- AI Mode: Uses a pre-trained neural network to predict control actions.
- Manual Mode: Allows user control via keyboard inputs

Our neural network processes 73 game-state features through 5 hidden layers (512-128-64-32 neurons) with ReLU activations and 20% dropout regularization. The model was trained on 112,919 samples using Adam optimization ($\text{lr}=0.007$) for 200 epochs, achieving a validation MAE of 0.0316. Preprocessing included standardization with parameters saved for real-time inference.

Input Features:

- Positional: DistanceFromStart, TrackPosition, Z-coordinate
- Dynamic: SpeedXYZ, WheelSpinVelocity, RPM
- Environmental: 19 track sensors, 36 opponent distances

Control Outputs:

- Acceleration [0-1]
- Braking [0-1]
- Clutch [0-1]
- Steering [-1,1] (normalized)

We selected a deep neural network (DNN) with supervised regression for our autonomous driving agent due to its superior ability to model complex, high-dimensional relationships between sensory inputs and control actions. This multi-layer perceptron architecture (512-128-64-32 neurons) with ReLU activations and dropout regularization was chosen over alternatives for several critical reasons:

Non-linear Pattern Capture:

Racing dynamics involve highly non-linear relationships between sensor inputs (73 features) and control outputs (4 actions)

DNNs excel at learning these complex mappings through hierarchical feature abstraction

Simpler models (linear regression, SVMs) fail to capture these intricate interactions

Multi-output Efficiency:

The unified architecture simultaneously predicts acceleration, braking, clutch, and steering

Shared hidden layers learn generalized representations across all controls
This is more efficient than training separate models for each control dimension

Robustness to Noise:

Dropout regularization (20%) prevents overfitting to training artifacts

The model generalizes better to unseen track conditions and opponent behaviors
This is crucial for handling TORCS' stochastic racing environment

Real-time Performance:

Once trained, forward propagation is extremely fast (μ s-level inference)

Meets TORCS' 50ms decision cycle requirement

More complex models (RNNs, transformers) introduce unacceptable latency

Data Efficiency:

Achieved low validation MAE (0.0316) with just 141k samples

Reinforcement learning alternatives would require orders of magnitude more environment interactions

Behavioral cloning provides optimal sample efficiency for expert trajectory replication

Compared to alternatives:

- Q-learning would require dangerous exploration crashes

- Genetic algorithms need impractical computation time

- Classic control systems can't adapt to dynamic opponents

Work Distribution:

When collecting telemetry data, we divided tracks such that each got 1.5 of a track to drive the car manually and get the data. Onto the training part, we didn't have much knowledge about ML data training so we were just checking different algorithms like I tried with RL but getting the environment deployed was really difficult. Atlast, through extensive research we found this one.